

Just when I thought I was out, they pull me back in.

Сильвио Данте

Алгоритмы и структуры данных

2025–2026 учебный год

SET 4. Домашняя работа

Сбалансированные деревья поиска

декабрь

1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31				
пн	вт	ср	чт	пт	сб	вс

Немного инструкций

Домашняя работа SET 4 содержит *Блок Р* «Задания на разработку» — задачи, связанные с реализацией сбалансированных деревьев поиска и других связанных алгоритмов.

Решения заданий Блока Р загружаются в систему CODEFORCES и проходят автоматизированное тестирование. Для загрузки нужно перейти на <https://dsahse25.contest.codeforces.com> и выбрать соответствующее соревнование. Доступ к соревнованию предоставлен по тем же учетным данным, что и к системе Яндекс.Контест.

Домашняя работа SET 4 содержит 6 обязательных задач в Блоке Р. Баллы, которые можно получить за их решение, распределены следующим образом:

Блок Р						
P1	P2	P3	P4	P5	P6	P7b
7	8	10	15	15	10	10

Задачи, помеченные “b” не являются обязательными — баллы за их решение относятся к *бонусным*. Подтверждение решений бонусных задач сопровождается обязательной *устной защитой*.

Важные даты

- 1. Домашняя работа SET 4 открыта с **18:00 01 декабря 2025 г.**
- 2. Прием решений завершается в **02:00 15 декабря 2025 г.**

Содержание

Задача P1	AVL-проверка	1
Задача P2	Бухгалтерия Bada Bing	3
Задача P3	AVL-дерево	5
Задача P4	Красно-черное дерево	8
Задача P5	В-дерево	11
Задача P6		13
Задача P7	Своппер	13

Успехов!

Задача Р1. AVL–проверка

Имя входного файла:	стандартный ввод
Имя выходного файла:	стандартный вывод
Ограничение по времени:	1 секунда
Ограничение по памяти:	256 мегабайт

После инцидента в Сосновых пустошах Тони одолевала тревога. Мысль о том, что Полли и Кристофер чуть не замерзли насмерть из-за одного «дизайнера интерьеров» (который, возможно, всё ещё бегает по лесу), вызвала у Тони очередной приступ панической атаки. Он осознал, что все его «деревья» контактов и операций разрослись слишком хаотично. Сидя в «Satriale's» над куском габагула, Тони объяснил задачу: иерархия должна быть идеально сбалансирована, чтобы ни один капо не брал на себя лишнего. В терминах вашего «бизнеса» это означает проверку на AVL-сбалансированность: разница высот поддеревьев любого узла не должна превышать единицы.



Дерево поиска должно быть реализовано в виде отдельной структуры `BinaryTree`, в которой поддерживается метод вставки.

Формат входных данных

На вход программа получает последовательность целых уникальных чисел (пусть n – длина последовательности, $0 \leq n \leq 2 \cdot 10^5$; числа, не превышающие по модулю 10^9). Последовательность завершается числом 0 (при этом число 0 в последовательность не входит). Постройте бинарное дерево поиска, соответствующее данной последовательности. Для корректного выполнения задания производить балансировку дерева для входной последовательности не надо.

Ознакомьтесь с примерами последовательностей в тестах и построенных для них деревьях в примечаниях для лучшего понимания условия.

Формат выходных данных

Определите, является ли дерево AVL-сбалансированным, выведите слово YES или NO соответственно.

Система оценки

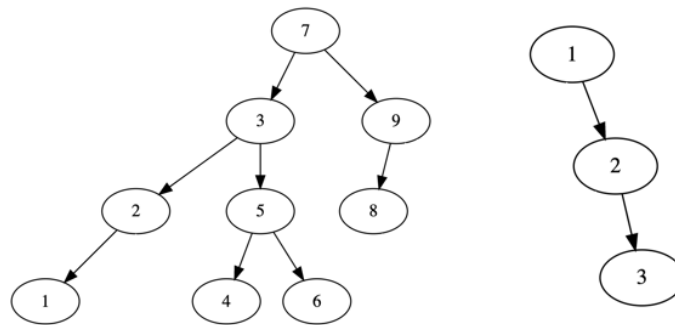
Подзадача	Баллы	Дополнительные ограничения	Необходимые подзадачи	Информация о проверке
0	—	тесты из условия	—	полная
1	1.5	$n \leq 100$	0	первая ошибка
2	1.5	$n \leq 10^3$	0–1	первая ошибка
3	2	$n \leq 10^5$	0–2	первая ошибка
4	2	$n \leq 2 \cdot 10^5$	0–3	первая ошибка

Примеры

стандартный ввод	стандартный вывод
7 3 2 1 9 5 4 6 8 0	YES
1 2 3 0	NO

Замечание

Слева показано дерево поиска, построенное для первого теста, справа — для второго:



Задача Р2. Бухгалтерия Bada Bing

Имя входного файла:	стандартный ввод
Имя выходного файла:	стандартный вывод
Ограничение по времени:	1 секунда
Ограничение по памяти:	256 мегабайт

Сильвио Данте заперся в задней комнате клуба «Bada Bing», пытаясь привести в порядок теневую бухгалтерию, пока Джорджи снова возится с ведром для льда. Чтобы сбить со следа налоговую и ФБР, Сил внедрил систему кодирования: «красные» узлы — это «опасные» деньги, а «чёрные» — доходы через легальный бизнес. У Тони есть жесткие требования к безопасности этой схемы: во-первых, верхушка (корень) всегда должна выглядеть чистой (быть чёрной), чтобы босс был вне подозрений. Во-вторых, нельзя допускать жадности: два «опасных» дела (красные узлы) никогда не должны идти подряд. И наконец, любой путь до самого низа должен проходить через одинаковое количество легальных этапов (высота по чёрным узлам). Ваша задача — проверить структуру книги Сильвио на соответствие RB-дереву.



В дереве обычно присутствуют фиктивные листовые вершины (`null`), которые по определению черные. Мы здесь будем придерживаться этого, но во входе ее цвет явно не задается.

Заметим, что черная вершина может иметь черного родителя.

Формат входных данных

В первой строке содержится целое число n ($0 \leq n \leq 10^6$) — размер дерева. Если $n = 0$, то на этом ввод закончен. Иначе на той же строке содержится натуральное число $root$ ($1 \leq root \leq n$) — номер корневой вершины.

В следующих n строках содержатся описания вершин. Каждая строка описания имеет следующий формат:

`number, key, left, right, color`, где

1. `number` — номер соответствующей вершины ($1 \leq number \leq n$, номера не повторяются);
2. `key` — ключ вершины ($-10^9 \leq key \leq 10^9$);
3. `left, right` — номера левого и правого сына, соответственно (`null`, если соответствующий сын — это фиктивная вершина; иначе $\in [1, n]$);

4. color – цвет вершины (R или B).

Гарантируется, что заданный граф – бинарное дерево.

Формат выходных данных

YES или NO в зависимости от того, выполняются ли условия сбалансированности красно-черного дерева (NO при $n = 0$).

Система оценки

Подзадача	Баллы	Дополнительные ограничения	Необходимые подзадачи	Информация о проверке
0	–	тесты из условия	–	полная
1	1	$n \leq 100$	0	первая ошибка
2	2	$n \leq 10^3$	0–1	первая ошибка
3	2	$n \leq 10^5$	0–2	первая ошибка
4	3	$n \leq 5 \cdot 10^5$	0–3	первая ошибка

Пример

стандартный ввод	стандартный вывод
5 1 1 2 2 4 B 2 0 null 3 R 3 1 null null B 4 3 null 5 R 5 4 null null B	NO

Задача Р3. AVL–дерево

Имя входного файла:	стандартный ввод
Имя выходного файла:	стандартный вывод
Ограничение по времени:	1 секунда
Ограничение по памяти:	256 мегабайт

Проект застройки эспланады приносит миллионы, но Ральфи Сифарето стал слишком много на себя брать, и это раздражает Нью-Йоркскую мафию. Тони понимает: стоит добавить ещё одну «фиктивную ставку» в перегруженную бригаду Ральфи, и хрупкий мир с семьёй Лупертацци рухнет под тяжестью их жадности. Чтобы «пирог» делился поровну Тони вводит жесткую систему контроля активов.



Ваша задача — реализовать механизм распределения конвертов на основе AVL-дерева. При добавлении нового участника в схему (вставка узла) вы должны мгновенно проверить баланс сил. Если одна из сторон становится слишком «тяжелой» (фактор баланса нарушен), необходимо провести «ротацию»: перераспределить потоки (выполнить левый или правый поворот), чтобы восстановить равновесие.

Ваш класс `AVLtree` должен содержать следующее:

1. Конструкторы и деструктор:

- (a) Конструктор по умолчанию
- (b) Деструктор

2. Методы:

- (a) Метод `insert(int value)` — Добавляет элемент `value` в дерево. Если такой элемент уже присутствует, тогда класс ничего не делает (не добавляет, не создает копии).
- (b) Метод `erase(int value)` — Удаляет элемент `value` из дерева. Если такого элемента нет в дереве, тогда класс ничего не делает.
- (c) Метод `find(int value)` — Возвращает указатель на значение, соответствующее элементу дерева, если оно в нем присутствует, и `nullptr` в противном случае.
- (d) Метод `traversal()` — Возвращает массив вершин по возрастанию их значений. Тип возвращаемого значения `int *`.

- (e) Метод `lowerBound(int x)` — Возвращает по заданному значению x указатель на минимальное значение y ($y \geq x$), соответствующее элементу дерева, или `nullptr`, если искомого элемента в дереве нет.
- (f) Метод `empty()` — Возвращает `true`, если дерево пустое, и `false` в противном случае.
- (g) Метод `getSize()` — Возвращает количество элементов в дереве.
- (h) Метод `getRoot()` — Возвращает ссылку на корень дерева типа `Node`.
- (i) Метод `getHeight()` — Возвращает высоту дерева, высота дерева начинается с 1.

Пример: если у нас есть только один элемент в дереве, и он содержит значение, тогда высота дерева равна 1.

Ваш класс должен состоять из `Node` узлов (тип элементов — `int`).
Структура `Node` должна содержать следующее:

1. Поле `value` — хранит значение текущего узла.
2. Поле `left` — хранит указатель на левый узел (поддерево).
3. Поле `right` — хранит указатель на правый узел (поддерево).
4. Поле `height` — хранит информацию о высоте узла (поддерева).

Формат входных данных

Вам нужно загрузить определение класса `AVLtree`. Загрузка неполного определения/реализации класса не гарантирует полное прохождение тестов в рамках группы. Файл с шаблоном всех функций `release.cpp` можно также найти в примечаниях по ссылке.

Формат выходных данных

Это задача-грейдер. Вашей целью будет загрузить реализацию функций класса `AVLtree`, шаблоны всех функций даны. **Не требуется загружать что-то иное, кроме реализации предоставленных функций класса.**

Например, вы не можете добавить свои поля, как-то модифицировать файл `avl_tree.h`. Тестироваться код будет с исходной версией этого файла, которая вам предоставлена.

Система оценки

Функция/Test	1	2	3	4	5	6	7	8	9	10	11
default	+	+	+	+	+	+	+	+	+	+	+
constructor	+	+	+	+		+	+	+	+	+	
destructor								+	+	+	
insert				+	+	+	+	+	+	+	+
erase			+	+	+		+	+	+	+	+
find	+	+	+	+	+	+		+	+	+	+
traversal					+	+	+		+		+
lowerBound				+		+				+	
empty	+	+	+					+	+	+	+
getSize			+							+	+
getRoot					+			+			
getHeight							+				+

Подзадача	Баллы	Дополнительная информация	Необходимые подзадачи	Информация о проверке
0	2	тесты конструктора, деструктора	—	полная
1	4	тесты основных методов	0	первая ошибка
2	4	дополнительные общие тесты	0–1	первая ошибка

Замечание

- Используйте шаблоны исходного кода, доступные в материалах задачи.
- Не разрешается добавлять свои приватные поля и функции в реализацию класса, это приведёт к ошибке компиляции.
- Метод `traversal()` : необходимо вернуть только сам массив, дополнительных манипуляций проводить не надо (удалять самостоятельно и т.д. и т.п.), так же не стоит использовать `std::vector<int>::data`.

Задача Р4. Красно-черное дерево

Имя входного файла:	стандартный ввод
Имя выходного файла:	стандартный вывод
Ограничение по времени:	1 секунда
Ограничение по памяти:	64 мегабайта

Цент бережет доллар, а для семьи важен любой доллар, поэтому вам необходимо реализовать новую систему хранения опасных и легальных доходов с повышенной точностью. Семья только недавно начала разбираться с компьютерами, поэтому не знает про floating-point rounding error. Но так как спорить с семьей дело небезопасное, вам в любом случае придется реализовать красно-чёрное дерево RBTree для хранения 32-битных целых чисел со знаком.



Ваш класс должен содержать следующее:

Конструкторы и деструктор:

1. Конструктор по умолчанию.
 - Создает пустое красно-черное дерево. `root == nullptr`.
2. Конструктор на списке инициализации.
 - Принимает на вход `std::initializer_list`, на значениях которого будет строиться красно-черное дерево.

3. Деструктор.

Методы:

1. Метод `insert`.
 - Добавляет ключ `key`.
 - Если красно-черное дерево пустое – обновляется `root`.

2. Константный метод **find** с указанием ключа – возвращает указатель на значение.
 - Если ключ отсутствует в красно-черном дереве, то возвращает **nullptr**.
3. Константный метод **size** – возвращает количество элементов в дереве.
4. Константный метод **lowerBound** по заданному значению – возвращает указатель на минимальное значение y , соответствующее элементу дерева, такое что $x \leq y$, или **nullptr**, если искомого элемента в дереве нет.
5. Константный метод **empty** - выполняет проверку дерева на пустоту.
 - Если дерево пустое, возвращаем **true**, иначе **false**.

Поля:

1. Публичное поле **root** – указывает на вершину красно-черного дерева.

В красно-черном дереве предусмотрен дополнительный атрибут узлов дерева — «цвета» (enum-класс **Color**). Этот атрибут может принимать одно из двух возможных значений:

1. **RED**.
2. **BLACK**.

Предлагаемый шаблон класса **Node** содержит следующее:

1. Поле **left** – хранит указатель на левый узел.
2. Поле **right** – хранит указатель на правый узел.
3. Поле **parent** – хранит указатель на родительский узел.
4. Поле **color** – хранит значение enum-класса **Color**.
5. Поле **key** – хранит ключ вашего узла.
6. Поле **height** – хранит высоту поддерева по чёрным вершинам.
7. Поле **size** – хранит количество узлов в поддереве.

Реализация конструкторов **Node** остаётся на ваше усмотрение.

Проверка вашего КЧД на правильность происходит обходом по всему дереву.

Система оценки

Группа	Баллы	Необх. группы	Комментарий
1	3	-	Тесты конструкторов
2	7	1	Тесты на основные методы
3	5	2	Дополнительные общие тесты

Замечание

В заголовочном файле **"rbtree.h"** в тестирующей системе уже определены классы **RBTree** и **Node**, а также **enum Node**. Исходный код этого файла доступен в материалах задачи. **Не забудьте подключить его с помощью `include "rbtree.h"`!**

Тестируемые методы

Test	std::initializer_list	constructor	insert	size	find	lowerBound	empty
1		+					+
2	+	+	+				+
3		+	+	+			
4		+	+	+			+
5		+	+	+			
6		+	+	+	+		
7		+	+	+		+	
8		+	+	+	+		
9		+	+	+		+	
10		+	+	+			+

Задача Р5. В-дерево

Имя входного файла:	стандартный ввод
Имя выходного файла:	стандартный вывод
Ограничение по времени:	1 секунда
Ограничение по памяти:	256 мегабайт

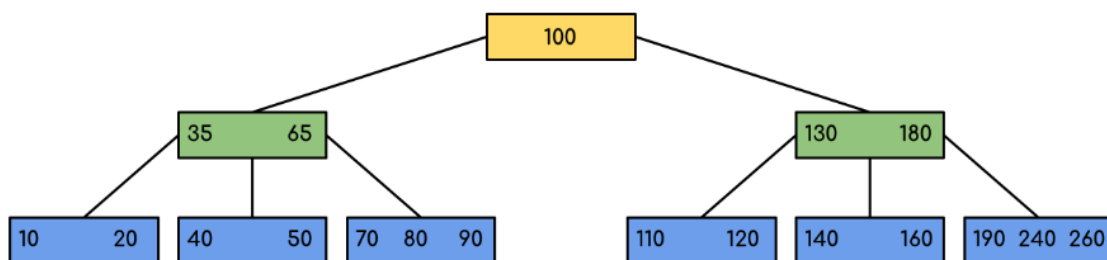
ФБР зажало семью в тиски. На стене в офисе агента Харриса висит огромная схема иерархии ДиМео — та самая, которую они используют для построения дела по закону RICO. Тони Сопрано, сидя в подсобке «Bada Bing», понимает: старая система «один капитан — два солдата» делает структуру слишком глубокой и неповоротливой. Чтобы передать приказ вниз, нужно сделать слишком много звонков, а каждый звонок — это риск прослушки. Тони решает реструктурировать семью по принципу В-дерева. Теперь каждый капо (узел) держит под собой целую бригаду (массив ключей), а не пару человек.



В-дерево создается на основе входной последовательности чисел длины N . Минимальная степень дерева t (не меньшая 2) определяет, какое количество узлов и потомков может содержать каждый узел дерева:

- Корень содержит от 1 до $2t - 1$ ключей, если дерево не пусто и от 2 до $2t$ детей при высоте большей 0.
- Все узлы, кроме корневого, содержат от $t - 1$ до $2t - 1$ ключей и от t до $2t$ сыновей.

Например, В-дерево со степенью $t = 2$ может быть представлено так:



Реализуемый класс `BTree` должен содержать следующие публичные методы:

1. Конструктор с параметром t (минимальной степени ветвления дерева).

2. Метод `insert(int key)`, который добавляет элемент в дерево. Если элемент уже присутствует, метод не должен ничего делать. Тип возвращаемого значения `void`.
3. Константный метод `size()`, возвращающий количество элементов в дереве. Например, для дерева выше это: 9.
4. Константный метод `sum()`, возвращающий сумму ключей во всех листовых вершинах дерева. Например, для дерева выше это: 1580.

Система оценки

Группа	Баллы	Доп. ограничения		Необх. группы	Комментарий
0	2	$N = 0$	—	—	Компиляция.
1	3	$N \leq 50$	$t \leq 2 \cdot 10^6$	0	
2	3	$N \leq 2 \cdot 10^4$	$t \leq 2 \cdot 10^3$	0	
3	4	$N \leq 10^5$	$t \leq 200$	0	
4	3	$N \leq 10^6$	$t \leq 20$	0 – 3	

Замечание

Вы должны отправить `.cpp` файл, **содержащий определение классов `BTree` и `Node`**. Файл с объявлением `BTree` и `Node` можно найти в прикрепленных материалах задачи. **Не забудьте добавит `#include "btree.h"`!**

Гарантируется, что минимальная степень t создаваемого В-дерева находится в пределах $2 \leq t \leq 2 \cdot 10^6$, числа по модулю не больше 10^9), а также, что методы `size()` и `sum()` вызываются единожды - в конце программы. Использование декораторов дерева для разделения ключей в вершинах не допускается.

Задача Р7b. Своппер

Имя входного файла:	стандартный ввод
Имя выходного файла:	стандартный вывод
Ограничение по времени:	1 секунда
Ограничение по памяти:	256 мегабайт

Последние пару месяцев в семье ДиМео царит полный хаос: то Фьюрио ломает бухгалтерскую программу, то Полли забудет пароль от сервера, то Кристофер «случайно» удалит полгода отчётов, пытаясь скачать очередной сценарий. А тут ещё Джонни Сэк, сидя у себя в Нью-Йорке, решил, что пора «модернизировать» совместный бизнес и прислал Тони новую железку — «центральный управляющий компьютер». Говорит, что вещь надёжная. Но программа к нему нужна специальная.



По словам Джонни Сэка, эта штука должна поддерживать структуру под названием «Своппер», которая помогает «перекладывать дела» между отделениями, чтобы у ФБР «глаза разбежались». К сожалению, если это не сделать, то Джонни Сэк опять позвонит ночью и начнёт рассказывать, как он «всё сделал бы сам, если бы не обстоятельства», так что вам придется это сделать. Итак, Своппер должен уметь следующее:

- Взять отрезок чётной длины от x до y и поменять местами число x с $x + 1$, $x + 2$ с $x + 3$, и т.д.
- Посчитать сумму чисел на произвольном отрезке от a до b .

Разработайте эффективную по вычислительной сложности реализацию Своппера.

Формат входных данных

Входные данные содержат один или несколько тестов. В первой строке каждого теста записаны число N — длина последовательности и число M — число операций ($1 \leq N, M \leq 100\,000$). Во второй строке теста содержится N целых чисел, не превосходящих 10^6 по модулю — сама последовательность. Далее следуют M строк — запросы в формате 1 x_i y_i — запрос первого типа, и 2 a_i b_i — запрос второго типа. Сумма всех N и M по всему файлу не превосходит 200 000. Файл завершается строкой из двух нулей. Гарантируется, что $x_i < y_i$, а $a_i \leq b_i$.

Формат выходных данных

Для каждого теста выведите ответы на запросы второго типа, как показано в примере. Разделяйте ответы на тесты пустой строкой.

Система оценки

Подзадача	Баллы	Дополнительные ограничения	Необходимые подзадачи	Информация о проверке
0	–	тесты из условия	–	полная
1	2	$N, M \leq 100$	0	первая ошибка
2	2	$N, M \leq 10^3$	0–1	первая ошибка
3	6	$N, M \leq 10^5$	0–2	первая ошибка

Пример

стандартный ввод	стандартный вывод
5 5 1 2 3 4 5 1 2 5 2 2 4 1 1 4 2 1 3 2 4 4 0 0	Swapper 1: 10 9 2